

VHDL Assignment #4: Getting Started with Altera DE1-SoC Board and Sequential Statements

1 Instructions

- TA in charge: Maninder Bir Singh Gulshan (maninder.gulshan@mail.mcgill.ca) - please utilize discussion boards on myCourses for questions as much as possible.
- Due date: Tuesday, November 7, 2023 by 11:59 pm EDT.
- Submission is in teams using myCourses (only one team member submits). In the report, provide the names and McGill IDs of the team members.
- Late submissions will incur penalties as described in the course syllabus.

2 Learning Outcomes

After completing this assignment you should know how to:

- Use CAD tools to design and implement a BCD to 7-segment LED decoder on the Altera DE1-SoC board
- Test digital circuits on the Altera DE1-SoC board
- Use the sliding switches on the Altera DE1-SoC board to specify the inputs to your circuits
- Use the 7-segment LED display on the Altera DE1-SoC board to display the output of your circuits
- Describe a circuit using sequential assignment statements

3 Introduction

In this assignment, you will use a previously designed BCD adder circuit, you will implement and test the circuit on the Altera DE1-SoC board. You will learn how to work with the Altera DE1-SoC board, use switches, and the 7-segment LED display.

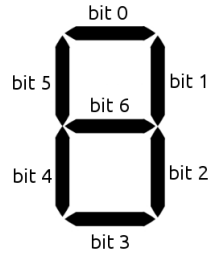
If you need any help regarding the lab materials, you can

- Ask the TA for help during lab sessions and office hours.
- Refer to the text book. In case you are not aware, Appendix A “VHDL Reference” provides detailed information on VHDL.
- You can also refer to the tutorial on Quartus and ModelSim provided by Intel ([click here for Quartus](#) and [here for ModelSim](#)).

It is highly recommended that you first try to resolve any issue by yourself (refer to the textbook and/or the multitude of VHDL resources on the Internet). Syntax errors, especially, can be quickly resolved by reading the error message to see exactly where the error occurred and checking the VHDL Reference or examples in the textbook for the correct syntax.

4 BCD to 7-Segment LED Decoder

A 7-segment LED display includes 7 individual LED segments, as shown below. By turning on different segments together, we can display characters or numbers. There are six of these displays on the DE1-SoC board, which you will use later to display the result of your full implementation of the adder.



We will need a circuit to drive the 7-segment LEDs on the DE1 board, called 7-segment decoder. It takes as input a 4-bit BCD-formatted code representing the 10 digits between 0 and 9, and generates the appropriate 7-segment display associated with the input code. For many LED displays, including the ones on the DE1 board, segments turn on when their segment inputs are driven low, that is “0” means on and “1” means off. This scheme for the inputs is called “active low.” The VHDL code for the 7 segment decoder is provided below.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity seven_segment_decoder is
port ( code          : in std_logic_vector(3 downto 0);
      segments_out   : out std_logic_vector(6 downto 0));
end seven_segment_decoder;

architecture decoder of seven_segment_decoder is
begin
WITH code SELECT
segments_out <=
"1000000" WHEN "0000", -- 0
"1111001" WHEN "0001", -- 1
"0100100" WHEN "0010", -- 2
"0110000" WHEN "0011", -- 3
"0011001" WHEN "0100", -- 4
"0010010" WHEN "0101", -- 5
"0000010" WHEN "0110", -- 6
"1111000" WHEN "0111", -- 7
"0000000" WHEN "1000", -- 8
"0010000" WHEN "1001", -- 9
"1111111" WHEN others;
end decoder;
```

In the next section, you will use the 7-segment LED decoder to display the inputs/outputs of the one-digit BCD adder.

5 Wrapper Design

In this part of the lab, you will design a circuit (that is called “wrapper” circuit) that performs addition of two 4-bit BCD-formatted inputs A and B, and displays the inputs as well as the result of the addition in BCD format using the 7-segment LEDs on the DE1-SoC board. You will need, therefore, to use four 7-segment LEDs on the board: the first two 7-segment LEDs will be used to display the inputs A and B, while the other two 7-segment LEDs will be used to display the result of the addition. We need two LED displays for the output as the result may be two BCD digits long. Note that you should use the binary-to-7-segment LED decoder to obtain appropriate 7-segment display codes.

Use the following entity declaration to write a VHDL description of the wrapper circuit. Note that you will need four instances of the `seven_segment_decoder` component and one instance of `firstname_lastname_bcd_adder` component in your VHDL description. You can use the BCD adder that one of the members of your group designed in the previous assignment.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity firstname_lastname_wrapper is
Port ( A, B      : in  std_logic_vector(3 downto 0);
      decoded_A  : out std_logic_vector(6 downto 0);
      decoded_B  : out std_logic_vector(6 downto 0);
      decoded_AplusB : out std_logic_vector(13 downto 0));
end firstname_lastname_wrapper;
```

where `firstname_lastname` in the name of the entity is the name of one of the students in your group.

The wrapper circuit has two 4-bit inputs, A and B, each representing a BCD digit. The outputs are: a 7-bit display code for A, a 7-bit display code for B, and two 7-bit display codes for the sum $A+B$.

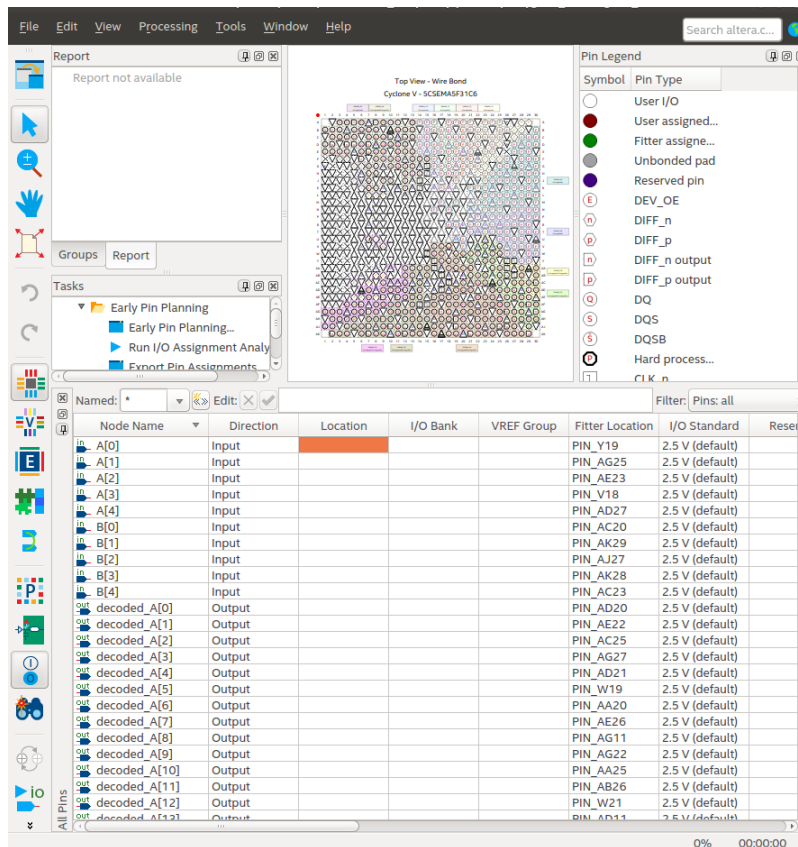
6 Testing the Wrapper on the Altera Board

You will now test the wrapper circuit you designed in Section 5. Follow the instructions described in Assignment #1 to create a project. Make sure to select the **Cyclone V** family of FPGAs, with the following part number: **5CSEMA5F31C6** when creating a project. Once created, import the VHDL description of the wrapper circuit and its components into the project and compile to make sure there are no syntax errors in your design. Once you have compiled the wrapper circuit, it is time to map it onto the target hardware, in this case the Cyclone V chip on the Altera DE1-SoC board.

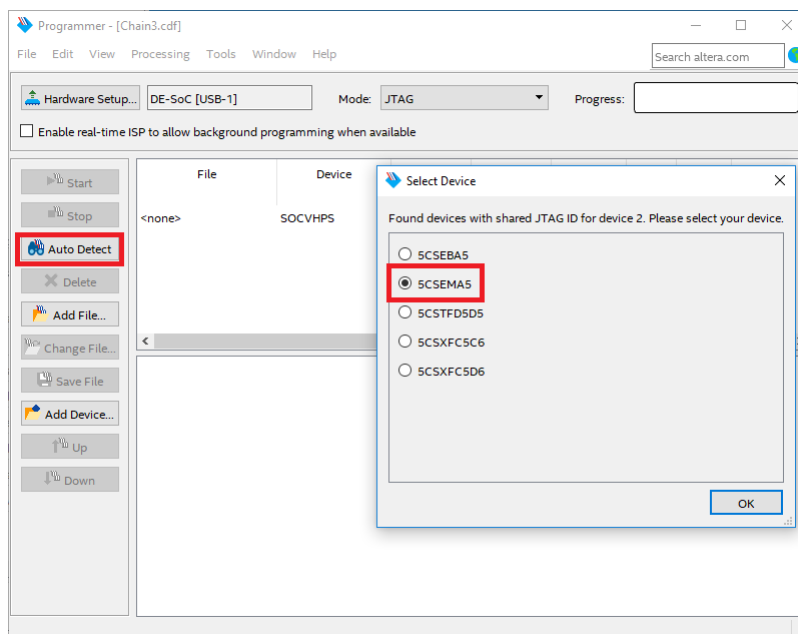
Since you will now be working with an actual device, you have to consider to which device package pins (physical pins) the various inputs and outputs of the project are connected. The FPGA chip on the board communicates with the outside world by sending and receiving binary signals using physical pins. You can think of these pins as the ports of an VHDL entity. Some of the input pins are connected to the sliding switches on the board, and some of the output pins are connected to the LED displays. In particular, you will want to connect the LED segment outputs from the instances of the `seven_segment_decoder` circuit (*i.e.*, the outputs of the wrapper circuit) to the corresponding pins of the four 7-segment LED displays on the board. See Section 3.6.2 of the DE1-SoC User Manual. The mapping segments of the each 7-segment LED on the board to the pins on the Cyclone FPGA device, is listed in Table 3-9 on page 27 of the DE1-SoC User Manual.

You will also want to connect, for testing purposes, four of the slide switches on the DE1-SoC board to the input A, and another four switches to the input B of the wrapper circuit. See Section 3.6.1 of the DE1-SoC User Manual. The mapping of the slide switches to the FPGA pins is listed in Table 3-6 on page 25 of the DE1-SoC user manual.

You must now notify the compiler to which pins the input and output signals of your wrapper circuit should be connected. For example, if `B[0]` is connected to switch SW0 (See Fig. 3-15 of the manual on page 24), you should notify the compiler that `B[0]` is assigned to pin `PIN_AB12`. You specify the pin assignments for your inputs and outputs by using the **Pin Planner**, which can be done by choosing the Pins item in the **Assignments** menu. See example below. Note that this figure is just an example, and does not correspond to the wrapper circuit that you are supposed to design in this assignment.



Once you have assigned all of the inputs and outputs of your circuit to appropriate device pins, re-compile your design. Your design is now ready to be downloaded to the target hardware. You may want to go over Section 4.1 of the DE1-SoC user manual for information on configuring (programming) the Cyclone V FPGA on the board. You will be using the JTAG mode to configure the device. Take the board out of the kit box, connect the USB cable to the USB port of the computer and to the USB connector on the board. Next, select the Programmer item from the Tools menu. Click Auto Detect and then select the correct device (5CSEMA5), as shown below. Both FPGA device and HPS should be detected.



Next, double-click the FPGA device (5CSEMA5), from the window that pops up, add the .sof file created by Quartus. Finally, check the “Program/configure” box beside the 5CSEMA5 device, and then click “Start”. See also Section “Configuring the FPGA in JTAG Mode” on p. 12 of the manual. Now, you should be able to use the slide switches to insert values for inputs A and B. The 7-segment LEDs should also display inputs and outputs in BCD format.

7 Sequential Assignment Statements

In the previous VHDL assignments, you have learned several types of assignment statements such as simple assignment statements, selected assignment statements, and conditional assignment statements. All of these statements have the property that the order in which they appear in the VHDL code does not affect the meaning of the code, that is, it does not affect the synthesized circuit. For this reason, these statements are usually referred to as concurrent assignment statements.

VHDL also has a second category of statements, called sequential assignment statements, for which the order of the statements may affect the meaning of the code. In the following, we briefly discuss this new kind of statements. *They are described in more detail in Sections 6.6.6 and 6.6.7 of the textbook. Please read these sections before you attempt this assignment.* Even more details can be found in Appendix A.9 of the textbook.

The two main types of sequential assignment statements are: if-then-else statements and case statements. These sequential assignment statements must be placed inside a block, called a *process* block. The PROCESS block starts with the keyword “PROCESS”. It has an optional label and a sensitivity list. Following the PROCESS keyword is the statement “BEGIN”. Any statement between “BEGIN” and the “END PROCESS label;” are sequential statements.

```
label: PROCESS (sensitivity list)
BEGIN
--sequential statements
END PROCESS label;
```

The sensitivity list contains signals. Unlike concurrent statements which are executed all the time, the process block is activated only when one of the signals in its sensitivity list changes its value. Once activated, the statements inside the process block are executed *sequentially* in the order they appear in the block. There are two things to note: (i) Any assignments made to signals inside the process are not visible outside the process until all of the statements in the process have been evaluated; (ii) in case of multiple assignments to the same signal, only the last one determines the final value of the signal. Also note that VHDL allows multiple processes to be described within the same architecture.

7.1 IF-THEN-ELSE Statements

IF-THEN-ELSE statements are used to modify the behavior of your function depending on whether one or more conditions hold. The syntax of IF-THEN-ELSE statements is shown below. Note that the “END IF” must be separated by a space.

```
IF condition THEN
-- sequential statements
ELSE
-- sequential statements
END IF;
```

You may have nested IF and ELSE as follows:

```
IF condition THEN
-- sequential statements
ELSIF condition THEN
-- sequential statements
ELSIF condition THEN
-- sequential statements
ELSE
-- sequential statements
END IF;
```

In this structure only one of the branches is executed depending on the condition. Even if there are several conditions which are true in this structure only the first TRUE condition will be followed. After the execution of the sequential statements within the first true condition the statements after the END IF will be executed next. Therefore it is very important to write the order of IF blocks and ELSIF blocks according to your desired behavior.

7.2 CASE Statements

CASE statements consider all of the possible values that an object can take and execute a different branch depending on the current value of the object as shown next.

```
CASE object IS
WHEN value1 =>
-- statements
WHEN value2 =>
-- statements
WHEN value3 =>
-- statements
-- etc.
WHEN OTHERS =>
-- statements
END CASE;
```

Note that the CASE statement must include a WHEN clause for each of the possible values of object. This necessitates a WHEN OTHERS clause if some of possible values of object are not covered by WHEN clauses.

8 4-Bit Comparator

Comparators are a useful type of arithmetic circuits that compare the relative sizes of two binary numbers. In this assignment you will implement a 4-bit comparator circuit that takes two 4-bit unsigned inputs A and B , and determines which one of the cases $A = (B + 1)$, $A < (B + 1)$, $A \leq (B + 1)$, $A > (B + 1)$, $A \geq (B + 1)$ holds. It should detect the occurrence of overflow when performing $B + 1$, i.e., detect if $B + 1$ requires more than 4 bits. Use the following entity declaration for your implementation of the comparator circuit.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity firstname_lastname_comparator is
Port ( A, B : in std_logic_vector(3 downto 0);
AgtBplusOne : out std_logic;
AgteBplusOne : out std_logic;
AltBplusOne : out std_logic;
AlteBplusOne : out std_logic;
AeqBplusOne : out std_logic;
overflow : out std_logic);
end firstname_lastname_comparator;
```

Note that in case of overflow when performing $B + 1$, the comparator circuit outputs 1 for the **overflow** signal while the remaining signals (i.e., **AgtBplusOne**, **AgteBplusOne**, **AltBplusOne**, **AlteBplusOne** and **AlteBplusOne**) are set to 0. Otherwise, the circuits outputs proper values for **AgtBplusOne**, **AgteBplusOne**, **AltBplusOne**, **AlteBplusOne** and **AeqBplusOne** signals according to $A > (B + 1)$, $A \geq (B + 1)$, $A < (B + 1)$, $A \leq (B + 1)$, $A = (B + 1)$, respectively, while the **overflow** signal is set to 0. For example, if $A = 9_{10}$ and $B = 5_{10}$ then both **AgtBplusOne**, **AgteBplusOne** should be 1, while the rest (including **overflow**) should be 0. The **firstname_lastname** in the name of the entity is the name of one of the students in your group.

To describe your comparator in VHDL, use sequential statements in a single process block. Once you have described your circuit in VHDL, you should test your design on the FPGA board. Similar to the VHDL Assignment #4, the inputs of the comparator circuit are provided using the slider switches; you will need to use eight slider switches. To visualize the output signals of the comparator circuit, use the LEDs located right above the slider switches on the FPGA board. Note that you will need to use six LEDs (one for each output signal). When an output signal is set to '1', the corresponding LED turns on; otherwise, it will be off. Afterwards, perform the pin assignments and program the FPGA by following the instructions provided in VHDL Assignment #4. Note that

the pin locations of LEDs are different from those of 7-segment LEDs. The pin assignments for the LEDs are given in Figure 3.17 on p. 25 of the Altera board manual. Once completed, test the functionality of your comparator circuit for different input values using the LEDs and slider switches.

9 Questions

1. Briefly explain your VHDL code implementation of all circuits.
2. Provide the VHDL code that you wrote for the wrapper circuit.
3. Show a representative photo of the board displaying the result of the addition of A and B, where A is the last (rightmost) digit of the McGill ID number of one of the students in your group and B is the last (rightmost) digit of the McGill ID number of the other student in the group (if there are three members in your group, choose two IDs and state which were chosen). Clearly indicate which 7-segment LEDs/sliding switches are assigned to which inputs/outputs of the circuit on the photo.
4. Report the number of pins and logic modules used to fit your designs on the FPGA board.
5. Given that $A = 5_{10}$, provide a **separate** simulation plot that demonstrates all possible cases for the 4-bit comparator, including a separate plot for the case where overflow occurs. A total of four plots should be included. Explain each plot and mark all inputs and outputs clearly.
6. Perform timing analysis (slow 1,100 mV model) of the 4-bit comparator and find the critical path(s) of the circuit. What is the delay of the critical path(s)?

10 Deliverables

You are required to submit the following deliverables on MyCourses. Please note that a single submission is required per group (by one of the group members).

- Lab report. The report should include the following parts: (1) Names and McGill IDs of group members, (2) an executive summary (short description of what you have done in this VHDL assignment), (3) answers to all questions in previous section (if applicable), (4) legible figures (screenshots) of schematics and simulation results, where all inputs, outputs, signals, and axes are marked and visible, (5) an explanation of the results obtained in the assignments (mark important points on the simulation plots), and (6) conclusions. Note - students are encouraged to take the reports seriously, points will be deducted for sloppy submissions. Please also note that even if some of the waveforms may look the same, you still need to include them separately in the report.
- Project files. Create a single .zip file named `VHDL#_firstname_lastname` (replace # with the number of the current VHDL assignment and `firstname_lastname` with the name of the submitting group member). The .zip file should include the working directory of the project.